

**TRƯỜNG ĐẠI HỌC CÔNG NGHỆ GIAO THÔNG VẬN TẢI**  
**KHOA CÔNG NGHỆ THÔNG TIN**



**BÁO CÁO NHÓM**  
**HỌC PHẦN: TRÍ TUỆ NHÂN TẠO**

**TÊN ĐỀ TÀI: BÀI TOÁN NGƯỜI DU LỊCH**

**GIÁO VIÊN HƯỚNG DẪN: TH.S PHẠM THỊ THUẬN**

**SINH VIÊN THỰC HIỆN: ĐÌNH HOÀNG ĐỨC**

**LƯU ĐỨC ANH DŨNG**

**BÙI VIỆT ĐẠT**

**TRẦN TRƯỜNG GIANG**

**NHÓM: 3**

**LỚP: 74DCHT23**

**HÀ NỘI 2024**

## This image shows a full page of white paper with horizontal dotted lines, typical of primary school writing paper. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

## BẢNG PHÂN CÔNG

| Họ và tên         | Công việc                          | Tiến độ công việc |
|-------------------|------------------------------------|-------------------|
| Lưu Đức Anh Dũng  | Tìm hiểu thuật toán và viết code   | 100%              |
| Đinh Hoàng Đức    | Tìm hiểu thuật toán và viết code   | 100%              |
| Trần Trường Giang | Tìm hiểu thuật toán và làm báo cáo | 100%              |
| Bùi Viết Đạt      | Tìm hiểu thuật toán và làm báo cáo | 100%              |

# MỤC LỤC

|                                                     |           |
|-----------------------------------------------------|-----------|
| <b>I. TỔNG QUAN.....</b>                            | <b>1</b>  |
| <b>1.1. Giới thiệu bài toán.....</b>                | <b>1</b>  |
| <b>1.2. Phân tích bài toán.....</b>                 | <b>1</b>  |
| - Dữ liệu đầu vào: .....                            | 1         |
| - Yêu cầu: .....                                    | 1         |
| - Đặc điểm: .....                                   | 1         |
| <b>II. THUẬT TOÁN.....</b>                          | <b>2</b>  |
| <b>2.1. Giới thiệu thuật toán .....</b>             | <b>2</b>  |
| <b>2.2. Tư tưởng của thuật toán .....</b>           | <b>3</b>  |
| <b>2.3. Minh họa của thuật toán .....</b>           | <b>4</b>  |
| <b>2.4. Đoạn mã chương trình và giải thích.....</b> | <b>10</b> |
| ✓ Đoạn mã chương trình:.....                        | 10        |
| ✓ Giải thích đoạn mã:.....                          | 12        |
| <b>III. KẾT QUẢ VÀ THỬ NGHIỆM.....</b>              | <b>16</b> |
| <b>3.1. Giao diện chương trình .....</b>            | <b>16</b> |
| <b>3.2. Kết quả thực thi.....</b>                   | <b>16</b> |
| <b>3.3. Phân tích hiệu suất .....</b>               | <b>17</b> |
| <b>IV. KẾT LUẬN VÀ ĐỀ XUẤT.....</b>                 | <b>18</b> |
| <b>V. TÀI LIỆU THAM KHẢO .....</b>                  | <b>18</b> |

# I. TỔNG QUAN

## 1.1. Giới thiệu bài toán

- Bài toán **Người du lịch** (hay Traveling Salesman Problem - TSP) là một trong những bài toán kinh điển trong lĩnh vực Toán học và Tin học. Mục tiêu của bài toán là tìm một chu trình (gọi là chu trình Hamilton) để một người du lịch ghé thăm **tất cả các thành phố** đúng một lần và quay lại thành phố ban đầu.

## 1.2. Phân tích bài toán

- **Dữ liệu đầu vào:**
  - Số lượng thành phố.
  - Ma trận kề
    - Có đường đi từ thành phố đến thành phố .
    - Không có đường đi.
- **Yêu cầu:**
  - Tìm một chu trình Hamilton (đi qua tất cả thành phố đúng một lần).
  - Chu trình phải quay về thành phố xuất phát.
- **Đặc điểm:**
  - Đồ thị không liên thông hoặc có đỉnh cô lập sẽ không có chu trình Hamilton.
  - Số lượng phương án tăng theo hàm giai thừa khi tăng.

## II. THUẬT TOÁN

### 2.1. Giới thiệu thuật toán

Thuật toán **quay lui** (Backtracking) là một kỹ thuật tìm kiếm theo kiểu thử-sai (**trial-and-error**) trong lập trình. Thuật toán này được sử dụng để giải quyết các bài toán đòi hỏi tìm kiếm tất cả hoặc một số giải pháp thoả mãn một tập hợp các ràng buộc.

- Xây dựng lời giải từng bước.
- Kiểm tra xem bước hiện tại có hợp lệ (thoả mãn ràng buộc) hay không.

**Nếu hợp lệ:** Tiếp tục mở rộng lời giải.

**Nếu không hợp lệ:** Quay lại bước trước đó (backtrack) để thử một lựa chọn khác.

- Tiếp tục quá trình cho đến khi tìm được lời giải hoặc thử hết tất cả các khả năng.

#### - Ý tưởng chính:

Xây dựng lời giải dần dần từng bước, duyệt qua không gian tìm kiếm.

Mỗi bước, kiểm tra xem:

**Có còn thoả mãn điều kiện không?**

Nếu không, "quay lui" (backtrack) để quay lại trạng thái trước đó và thử nhánh khác.

Lặp lại cho đến khi:

Tìm được tất cả các lời giải thoả mãn.

Hoặc xác định rằng không có lời giải.

#### - Cách hoạt động:

**Bắt đầu từ điểm khởi đầu** và thử từng khả năng tại đó.

**Kiểm tra ràng buộc:** Với mỗi khả năng thử, kiểm tra xem nó có thoả mãn điều kiện hay không.

**Tiếp tục hoặc quay lui:**

Nếu khả năng đó hợp lệ, tiếp tục tìm kiếm sâu hơn (mở rộng lời giải).

Nếu không hợp lệ, quay trở lại trạng thái trước đó và thử khả năng khác.

**Lặp lại cho đến khi:**

Tìm ra lời giải.

Hoặc thử hết các khả năng.

**- Ứng dụng:**

Thuật toán quay lui thường được sử dụng để giải quyết các bài toán liên quan đến:

**Tìm kiếm và liệt kê tất cả các lời giải:**

Bài toán **Tổ hợp** (Combinations) và **Hoán vị** (Permutations).

Bài toán **Tổ hợp tổ tiên/thử tất cả trường hợp** (Subset Sum).

**Đồ thị:**

- ✓ Bài toán **Chu trình Hamilton** và **Đường đi Euler**.
- ✓ Bài toán **Tô màu đồ thị**.
- **Xếp hình, trò chơi, câu đố:**
  - ✓ Bài toán **N-queens** (N quân hậu).
  - ✓ Sudoku, trò chơi mê cung.
- **Quy hoạch:**
  - ✓ Bài toán chia bài toán lớn thành các bài toán con để tìm kiếm lời giải.

**2.2. Tư tưởng của thuật toán****Bước 1: Xác định bài toán**

- Xác định bài toán cần giải, các điều kiện ràng buộc, và mục tiêu tìm kiếm (ví dụ: tìm tất cả lời giải hoặc tìm một lời giải duy nhất).

**Bước 2: Khởi tạo dữ liệu**

- Tạo cấu trúc dữ liệu cần thiết để lưu trữ trạng thái (ví dụ: mảng, danh sách, biến đánh dấu...).
- Xác định trạng thái ban đầu (ví dụ: bắt đầu từ điểm khởi đầu hoặc trạng thái ban đầu của lời giải).

**Bước 3: Xây dựng hàm đệ quy (Try)**

- Viết một hàm đệ quy thực hiện việc thử nghiệm từng khả năng:

**1. Xác định điều kiện dừng:**

- Nếu đạt trạng thái cuối (đã hoàn thành lời giải hoặc đã thử hết các khả năng), kiểm tra:
  - ✓ Nếu lời giải hợp lệ → Lưu lại hoặc in ra.
  - ✓ Nếu không hợp lệ → Kết thúc nhánh này và quay lui.

**2. Duyệt qua các khả năng:**

- Thử tất cả các lựa chọn khả thi tại bước hiện tại.
- Với mỗi lựa chọn:
  - ✓ Kiểm tra xem lựa chọn đó có hợp lệ (thỏa mãn ràng buộc) hay không.
  - ✓ Nếu hợp lệ:
    - Cập nhật trạng thái (tiến thêm một bước).
    - Gọi lại hàm đệ quy cho bước tiếp theo.
  - ✓ Nếu không hợp lệ:
    - Bỏ qua và thử lựa chọn khác.

### 3. Quay lui:

- Sau khi thử xong hoặc lời giải không hợp lệ, hoàn tác trạng thái (quay lui) để thử nhánh khác.

### Bước 4: Triển khai thuật toán

- Gọi hàm đệ quy từ trạng thái ban đầu, thường là Try(1) hoặc Try(0).

### Bước 5: Xuất kết quả

- Nếu tìm được lời giải: Xuất các lời giải thỏa mãn.
- Nếu không tìm được lời giải: Thông báo không có lời giải.

## 2.3. Minh họa của thuật toán

VD1:

Dữ liệu đầu vào: File DULIEU.INP có dạng:

```

8
0 1 0 0 0 0 1
1 0 1 0 0 0 0
0 1 0 1 1 0 0
0 0 1 0 0 1 1
0 0 1 0 0 1 0
0 0 0 1 1 0 1
0 0 0 1 0 1 0
1 0 0 0 0 1 0

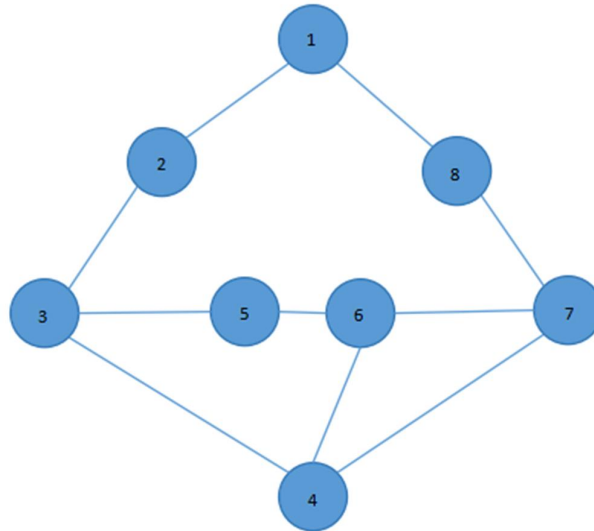
```

Kết quả đầu ra: File KETQUA.OUT lưu chu trình:

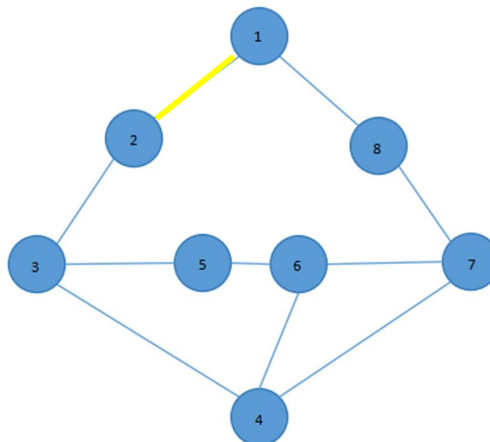
```

1 2 3 5 6 4 7 8 1
1 8 7 4 6 5 3 2 1

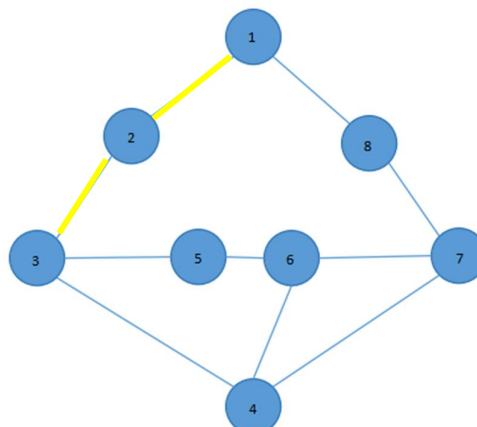
```

**Sơ đồ**

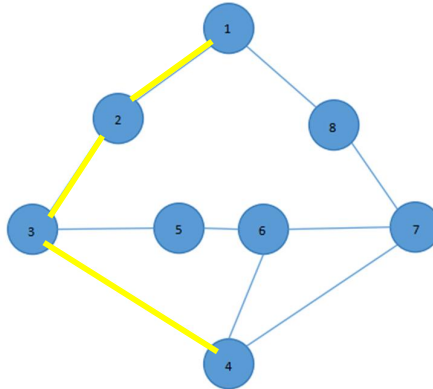
➤ Đỉnh bắt đầu: 1.



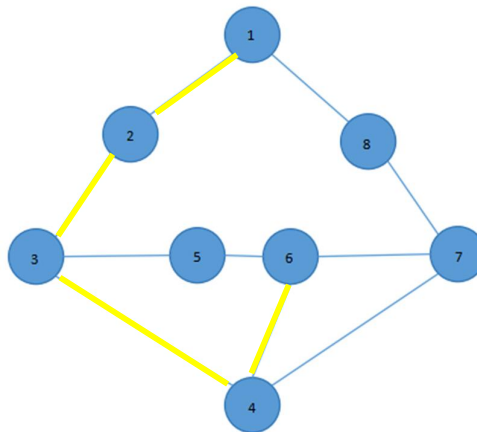
- Xét các đỉnh kề của 1:
- Đỉnh kề: 2, 8.
- Chọn đỉnh 2 (đỉnh đầu tiên hợp lệ chưa được thăm).



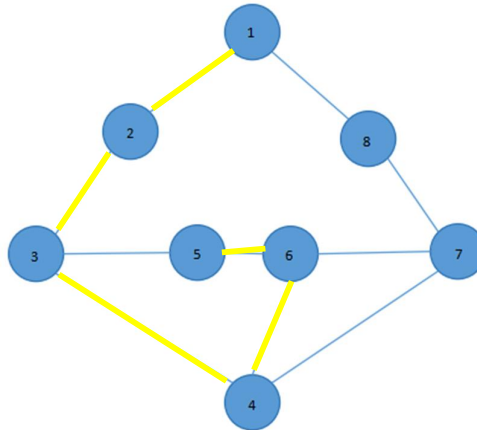
- Tại đỉnh 2, xét các đỉnh kề:
- Đỉnh kề: 1, 3.
- Đỉnh 1 đã thăm, chọn đỉnh 3.



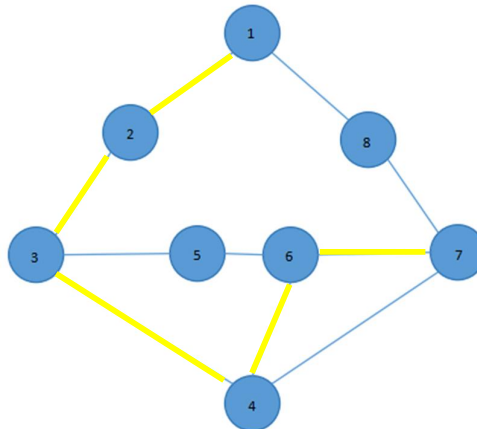
- Tại đỉnh 3, xét các đỉnh kề:
- Đỉnh kề: 2, 4, 5.
- Đỉnh 2 đã thăm, chọn đỉnh 4.



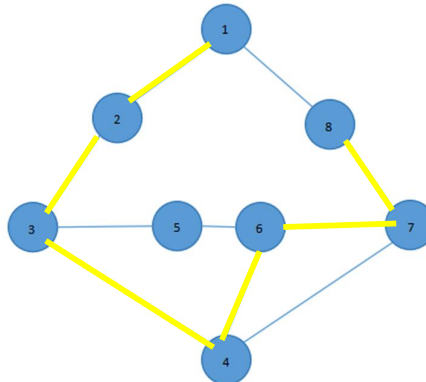
- Tại đỉnh 4, xét các đỉnh kề:
- Đỉnh kề: 3, 6, 7.
- Đỉnh 3 đã thăm, chọn đỉnh 6.



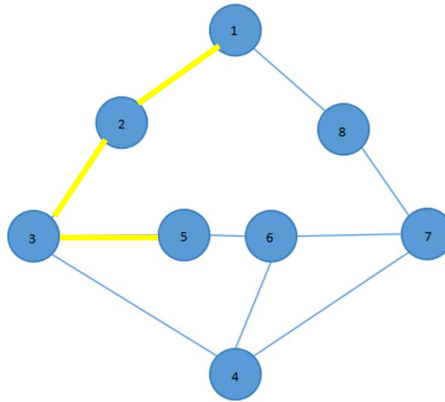
- Tại đỉnh 6, xét các đỉnh kề:
- Đỉnh kề: 4, 5, 7.
- Đỉnh 4 đã thăm, chọn đỉnh 5.



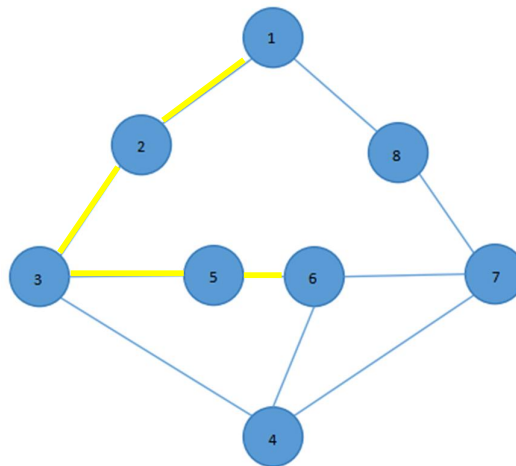
- Tại đỉnh 5, xét các đỉnh kề:
- Đỉnh kề: 3, 6.
- Tất cả các đỉnh kề đã được thăm, backtrack về đỉnh 6.
- Quay lại đỉnh 6, chọn đỉnh 7 thay vì 5.



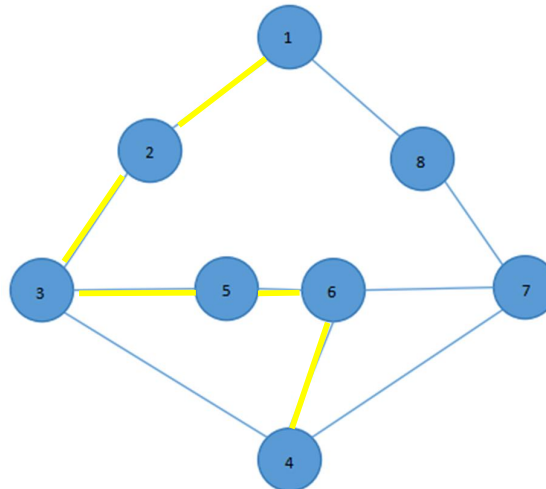
- Tại đỉnh 7, xét các đỉnh kề:
- Đỉnh kề: 4, 6, 8.
- Các đỉnh 4, 6 đã thăm, chọn đỉnh 8.



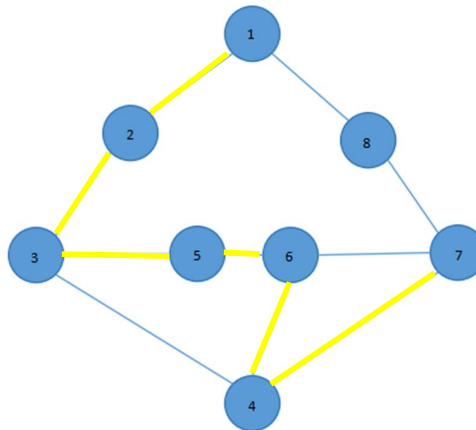
- Tại đỉnh 8, xét các đỉnh kề:
- Đỉnh kề: 1, 7.
- Tất cả các đỉnh kề đã được thăm, backtrack về đỉnh 7.
- Backtrack về đỉnh 6.
- Backtrack về đỉnh 4.
- Backtrack về đỉnh 3.
- Chọn đỉnh 5.



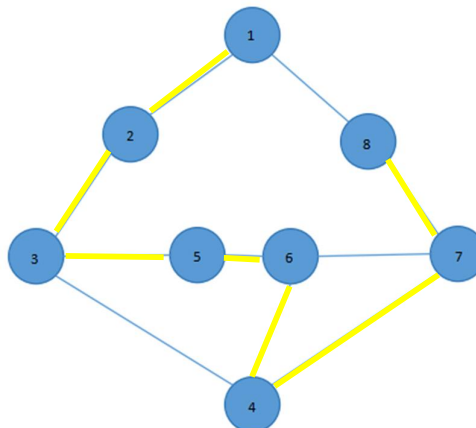
- Tại đỉnh 5, xét các đỉnh kề:
- Đỉnh kề: 3,6.
- Đỉnh 3 đã thăm, chọn đỉnh 6.



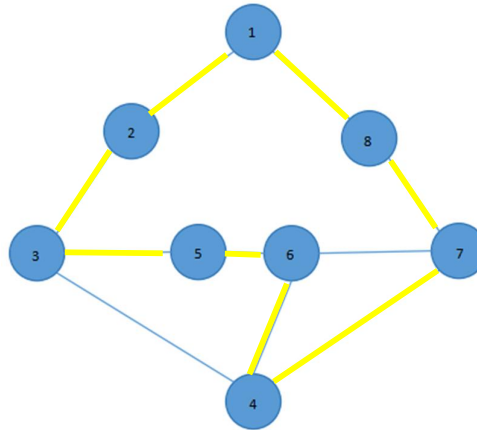
- Tại đỉnh 6, xét các đỉnh kề:
- Đỉnh kề: 4, 5, 7.
- Chọn đỉnh 4.



- Tại đỉnh 4, xét các đỉnh kề:
- Đỉnh kề: 3, 6, 7.
- Đỉnh 3, 6 đã thăm, chọn đỉnh 7.



- Tại đỉnh 7, xét các đỉnh kề:
- Đỉnh kề: 4, 6, 8.
- Đỉnh 4, 6 đã thăm, chọn đỉnh 8.



- Tại đỉnh 8, xét các đỉnh kề:
- Đỉnh kề: 1, 7.
- Đỉnh 7 đã thăm, chọn đỉnh 1.

==> *Tìm thấy đường đi: 1-2-3-5-6-4-7-8-1.*

## 2.4. Đoạn mã chương trình và giải thích

### ✓ Đoạn mã chương trình:

```
#include <bits/stdc++.h>

using namespace std;

int n, A[105][105], X[100];

bool visited[105];

bool foundHamilton = false; // Đánh dấu nếu tìm thấy chu trình Hamilton

void nhap() {
    cin >> n;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            cin >> A[i][j];
        }
    }
    memset(visited, false, sizeof(visited));
}
```

```
// Hàm thử để tìm chu trình Hamilton
void Try(int i) {
    for (int j = 1; j <= n; j++) {
        if (!visited[j] && A[X[i - 1]][j] == 1) { // Kiểm tra tính hợp lệ
            visited[j] = true;
            X[i] = j;
            cout << "Di chuyển đến đỉnh: " << j << " (bước " << i << ")" << endl; // In trạng thái hiện tại

            if (i == n) { // Nếu đã đi qua tất cả các thành phố
                if (A[X[i]][X[1]] == 1) { // Kiểm tra đường quay lại thành phố xuất phát
                    foundHamilton = true;
                    // In chu trình Hamilton
                    for (int k = 1; k <= n; k++) {
                        cout << X[k] << " ";
                    }
                    cout << X[1] << endl; // Quay lại đỉnh đầu
                    return; // Kết thúc khi tìm thấy một chu trình
                }
            } else {
                Try(i + 1); // Thử tiếp thành phố tiếp theo
            }
            visited[j] = false; // Backtracking
            cout << "Quay lại khỏi đỉnh: " << j << " (bước " << i << ")" << endl;
        }
    }
}

int main() {
```

```
// Đọc dữ liệu từ file DULIEU.INP
freopen("DULIEU.INP", "r", stdin);
freopen("KETQUA.OUT", "w", stdout);
nhap();
X[1] = 1;      // Bắt đầu từ thành phố 1
visited[1] = true; // Đánh dấu thành phố 1 đã được thăm
cout << "Bat dau tai dinh: 1" << endl;
Try(2);        // Bắt đầu tìm từ thành phố thứ 2
if (!foundHamilton) {
    cout << "Khong ton tai loi giai" << endl;
}
return 0;
}
```

✓ **Giải thích đoạn mã:**

```
int n, A[105][105], X[100];
bool visited[105];
```

- N là số thành phố.
- Mảng A để lưu thông tin đường đi, giả sử có đường đi từ thành phố 1 đến 2  
=> A[1][2]=1.
- Mảng X dùng để lưu đường đi.
- Mảng bool visited dùng để kiểm tra thành phố đã đi qua chưa.

```
bool foundHamilton = false; // Đánh dấu nếu tìm thấy chu trình Hamilton
```

```

void nhap() {
    cin >> n;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            cin >> A[i][j];
        }
    }
    memset(visited, false, sizeof(visited));
}

```

- Hàm nhap() dùng để đưa dữ liệu từ file “DULIEU.INP”
- Hàm memset dùng để cho toàn bộ mảng visited thành false.

```

void Try(int i) {
    for (int j = 1; j <= n; j++) {
        if (!visited[j] && A[X[i - 1]][j] == 1) { // Kiểm tra tính hợp lệ
            visited[j] = true;
            X[i] = j;
            if (i == n) { // Nếu đã đi qua tất cả các thành phố
                if (A[X[i]][X[1]] == 1) { // Kiểm tra đường quay lại thành
                    phố xuất phát
                        foundHamilton = true;
                        // In chu trình Hamilton
                        for (int k = 1; k <= n; k++) {
                            cout << X[k] << " ";
                        }
                        cout << X[1] << endl; // Quay lại đỉnh đầu
                        return; // Kết thúc khi tìm thấy một chu trình
                    }
                } else {
                    Try(i + 1); // Thử tiếp thành phố tiếp theo
                }
                visited[j] = false; // Backtracking
            }
        }
    }
}

```

- void Try(int i) : Hàm **Try** là hàm đệ quy, kiểm tra từng khả năng đi qua các đỉnh (thành phố) để tìm chu trình Hamilton. Biến i đại diện cho thứ tự của đỉnh hiện tại trong chuỗi.
- for (int j = 1; j <= n; j++) {} : Duyệt qua tất cả các đỉnh (thành phố) từ 1 đến n để kiểm tra xem có thể đi từ đỉnh hiện tại đến đỉnh j hay không.
- if (!visited[j] && A[X[i - 1]][j] == 1) {} :
  - o Kiểm tra điều kiện:
  - o !visited[j]: Đỉnh j chưa được đi qua.
  - o A[X[i - 1]][j] == 1: Có đường đi từ đỉnh trước đó (X[i-1]) đến đỉnh j.
  - o Nếu điều kiện thỏa mãn, tức là có thể đi đến đỉnh j.
- visited[j] = true; X[i] = j; :
  - o Đánh dấu đỉnh j là đã được thăm (visited[j] = true).
  - o Ghi nhận đỉnh j vào vị trí thứ i trong chu trình Hamilton (X[i] = j).
- if (i == n) {} : Kiểm tra nếu đã đi qua đủ n đỉnh, tức là đang ở bước cuối cùng của chu trình Hamilton.
- if (A[X[i]][X[1]] == 1) {} : Kiểm tra xem từ đỉnh cuối cùng có đường quay lại đỉnh xuất phát (X[1]) hay không. Đây là điều kiện để hoàn thành chu trình Hamilton.
- foundHamilton = true; : Đặt biến foundHamilton thành true để đánh dấu đã tìm thấy một chu trình Hamilton.
- cout << X[k] << " " : In ra kết quả.
- cout << X[1] << endl; : Thêm đỉnh đầu tiên vào cuối.
- } else { Try(i + 1); } : Nếu chưa đi qua đủ các đỉnh (i != n), gọi đệ quy để kiểm tra bước tiếp theo (i + 1).
- visited[j] = false; : **Backtracking**: Sau khi thử xong một lựa chọn, hoàn tác (đánh dấu lại đỉnh j là chưa thăm) để thử các khả năng khác.

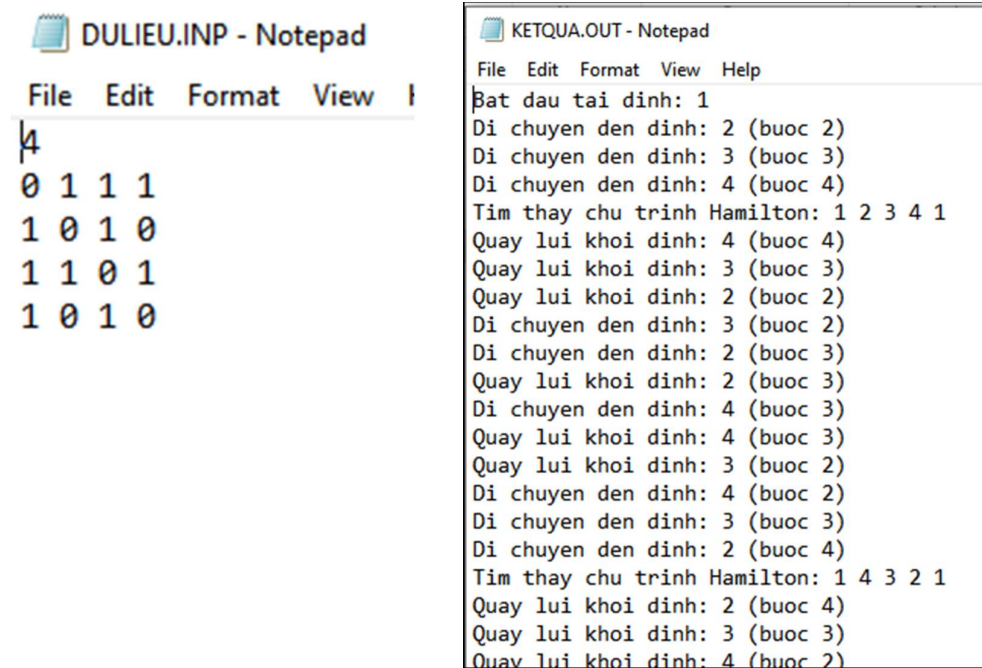
```
int main() {  
    // Đọc dữ liệu từ file DULIEU.INP  
    freopen("DULIEU.INP", "r", stdin);  
    freopen("KETQUA.OUT", "w", stdout);  
    nhap();  
    X[1] = 1;    // Bắt đầu từ thành phố 1  
    visited[1] = true; // Đánh dấu thành phố 1 đã được thăm  
    Try(2);    // Bắt đầu tìm từ thành phố thứ 2  
    if (!foundHamilton) {  
        cout << "Khong ton tai loi giai" << endl;  
    }  
}
```

- Hàm freopen dùng để lấy dữ liệu từ file “DULIEU.INP”.
- Hàm freopen dùng để nhập kết quả lại vào file “KETQUA.OUT”.
- Chạy hàm nhap() để nhập thông tin.
- X[1]=1 gán thành phố bắt đầu bằng 1.
- Visited[1] = true : đánh dấu thành phố 1 đã được thăm.
- Try(2): bắt đầu tìm đường đi từ thành phố 2.
- If() : Kiểm tra có tồn tại đường đi không, nếu không tồn tại đường đi thì kết luận: “Không tồn tại lời giải”.

### III. KẾT QUẢ VÀ THỬ NGHIỆM

#### 3.1. Giao diện chương trình

Chương trình được thiết kế với giao diện đơn giản nhưng trực quan. Dữ liệu đầu vào là một ma trận kề được nhập từ file, và kết quả được ghi và hiển thị trên file đầu ra.



#### 3.2. Kết quả thực thi

Chương trình đã được thử nghiệm với nhiều tập dữ liệu đầu vào khác nhau. Ví dụ 2:

- **Dữ liệu đầu vào:**

4

0 1 1 1

1 0 1 0

1 1 0 1

1 0 1 0

- **Kết quả đầu ra:**

- Chu trình Hamilton tìm được: 1 -> 2 -> 3 -> 4 -> 1

Thử nghiệm trên các tập dữ liệu lớn hơn cho thấy chương trình hoạt động hiệu quả đối với các bài toán nhỏ (). Tuy nhiên, với lớn hơn, thời gian tính toán tăng đáng kể.

### 3.3. Phân tích hiệu suất

- **Độ phức tạp:** Thuật toán có độ phức tạp thời gian , do đó không phù hợp với các bài toán lớn.
- **Thời gian chạy thực tế:**
  - o Với ví dụ ma trận vuông 8x8 : Thời gian chạy 0.1931 giây.

```

1 8
2 0 1 0 0 0 0 0 1
3 1 0 1 0 0 0 0 0
4 0 1 0 1 1 0 0 0
5 0 0 1 0 0 1 1 0
6 0 0 1 0 0 1 0 0
7 0 0 0 1 1 0 1 0
8 0 0 0 1 0 1 0 1
9 1 0 0 0 0 0 1 0
10

```

```

C:\Users\Asus\Desktop\ttnt\T x + v
-----
Process exited after 0.1931 seconds with return value 0
Press any key to continue . . . |

```

- o Với ví dụ ma trận vuông 12x12 : Thời gian chạy tăng lên 1.405 giây. Tăng gần gấp 10 lần so với ma trận vuông 8x8.

```

12
1 1 0 1 1 1 0 0 0 1 0 0
0 1 1 0 1 1 1 0 0 1 0 1
0 1 0 0 1 1 1 1 0 1 1 0
0 0 1 0 1 0 0 0 0 0 0 0
0 1 0 0 1 1 1 1 0 1 1 0
1 1 1 1 1 1 0 0 1 1 0 1
0 1 1 1 1 0 0 0 0 1 0 0
1 1 0 1 0 0 1 1 0 1 1 1
1 1 1 1 1 1 1 1 0 1 0 1
1 1 0 1 0 0 1 1 0 1 0 0
1 1 0 0 1 0 1 1 1 0 0 1
1 1 1 1 1 1 1 0 0 0 0 0

```

```

C:\Users\Asus\Desktop\ttnt\T x + v
-----
Process exited after 1.405 seconds with return value 0
Press any key to continue . . . |

```

- **Hạn chế:** Do tính chất duyệt tất cả các khả năng, thuật toán không khả thi khi quá lớn.

## IV. KẾT LUẬN VÀ ĐỀ XUẤT

1. **Kết luận:** Thuật toán quay lui là giải pháp hiệu quả cho bài toán TSP khi số lượng đỉnh nhỏ. Tuy nhiên, với các bài toán lớn, cần sử dụng các thuật toán tối ưu hơn.

2. **Đề xuất:**

- Sử dụng thuật toán nhánh cận (Branch and Bound) để giảm không gian tìm kiếm.
- Áp dụng các thuật toán heuristic như Genetic Algorithm hoặc Ant Colony Optimization để giải quyết bài toán TSP lớn.

## V. TÀI LIỆU THAM KHẢO

- GeeksforGeeks, Stack Overflow.
- Tài liệu giáo trình về thuật toán.
- Bài báo khoa học trên IEEE.